14-customizing-test-doubles/01/configuring_responses.rb
```
extractor = double('TwitterURLExtractor')

allow(extractor).to receive(:extract_urls_from_twitter_firehose)
  .and_yield('https://rspec.info/',   93284234987)
  .and_yield('https://github.com/',   43984523459)
  .and_yield('https://pragprog.com/', 33745639845)
```

We've chained together three and_yield calls. When the code we're testing calls extract_urls_from_twitter_firehose with a block, the method will yield to the block three times. Each time, the block will receive a URL and a numeric tweet ID.

Raising Exceptions Flexibly

When you're testing exception-handling code, you can raise exceptions from your test doubles using the and_raise modifier. This method has a flexible API that mirrors Ruby's raise method.[2] That means all of the following calls will work:

14-customizing-test-doubles/01/configuring_responses.rb
```
allow(dbl).to receive(:msg).and_raise(AnExceptionClass)
allow(dbl).to receive(:msg).and_raise('an error message')
allow(dbl).to receive(:msg).and_raise(AnExceptionClass, 'with a message')

an_exception_instance = AnExceptionClass.new
allow(dbl).to receive(:msg).and_raise(an_exception_instance)
```

In the examples we've shown you so far, we've been working with pure test doubles. These doubles have to be told exactly how to respond, because they don't have an existing implementation to modify.

Partial doubles are different. Since they begin as a real object with real method implementations, you can base the fake version on the real one. Let's look at how to do so.

Falling Back to the Original Implementation

When you're using a partial double to replace a method, sometimes you only want to replace it conditionally. You may want to use a fake implementation for certain parameter values but fall back on the real method the rest of the time. In these cases, you can expect or allow twice: once like you normally would, and once with and_call_original to provide the default behavior.

14-customizing-test-doubles/01/configuring_responses.rb
```
# fake implementation for specific arguments:
allow(File).to receive(:read).with('/etc/passwd').and_raise('HAHA NOPE')

# fallback:
allow(File).to receive(:read).and_call_original
```

2. https://ruby-doc.org/core-2.4.1/Kernel.html#method-i-raise